ACRME UML Class Diagrams — Design-First Summary

Author: Vishnuvardhan Reddy
Date: August 22, 2026
Status: Design-first tool (70% complete) — identifies gaps for focused design sessions
Purpose: Visual domain model, operation tracking, service architecture, and state machines

> **v2.2 reconciliation note (2 Sep 2026).** This summary is reconciled to Requirements Baseline v2.2 and the updated ADRs: the quota model is a **single governed pool** with logical earmarks (two-group only as an exception topology, QUA-004/ADR-002); three domain entities are added — `CustomerSeedRecord`, `SourceDestinationDRIndex`, `CVALEarmarkRecord`; the readiness verdict is the machine-readable enum `READY | READY_WITH_RISK | QUOTA_DEFICIT | RESERVATION_DEFICIT | CAPACITY_UNAVAILABLE | STALE_STATE | POLICY_BLOCKED | VALIDATION_REQUIRED` (RDY-002); the engine mode machine is five-state (`STEADY_STATE → DR_DECLARATION_PENDING → DR_EVENT_ACTIVE → FAILBACK_PENDING → STEADY_STATE`, with `INCIDENT_HOLD`). Authoritative schemas live in the FDD (`acrme_functional_design_document.md`) and TDD (`acrme_technical_design_document.md`).
>
> **v2.4 reconciliation note (7 Sep 2026).** This summary is further reconciled to Requirements Baseline **v2.4** and the core domain model (`uml_01_core_domain_model.md`). No entity is removed; the following v2.4 additions are folded into Diagram 1: (a) **per-AZ + regional CRG structure** — the earlier "three-CRG (Prod/NonProd/DR) per region" model is extended by **CAP-023** so each environment carries one **regional** CRG (`crg-<env>-<region>-reg`) plus one CRG **per AZ** (`az1/az2/az3`); `CapacityReservationGroup` gains `environment` and `crgScope` (`REGIONAL | ZONAL`) and an `isZonal()` accessor; (b) **seed-at-0 governance** — new entity `SeedMatrixEntry` (**CAP-022**, extends CAP-009) records the product-team-governed seed matrix initialised at 0, and `CapacityReservation` gains `reservationType` (`SEED | ACTIVE`) with `isSeed()`; (c) **reservation eligibility** — new entity `ReservationEligibility` (**CAP-020**/**CAP-021**) captures Availability-Set-VM ineligibility (CAP-020) and the deallocate/redeploy-to-AZ onboarding precondition (CAP-021), and `CapacityReservation` gains `placementType`; (d) **reactive discovery** — **CAP-024** auto-creates missing SKU/AZ reservations against the seed matrix and raises a scope-file governance item (reconciles CAP-019); (e) **even zone distribution** — **PLC-011** even ≈ `1/zone_count` distribution + rebalancing (tolerance C-13, formula Appendix A.9). Enumerations added: `ReservationType`, `CRGScope`, `PlacementKind`, `GovernanceStatus`. The **region model is geography-aware** (US three-region Prod/CVAL/DR distinct; EU/Australia/Asia Pacific two-region with CVAL+DR co-located per PLC-010a; Middle East `DR_NOT_OFFERED` per DEC-001) — see ADR-001 v2.4. Authoritative schemas remain in the FDD and TDD.

Executive Summary

This document presents **four comprehensive UML diagram sets** for the Azure Capacity Reservation Management Engine (ACRME). These diagrams serve as a **design-first tool** to:

1. **Visualize the domain model** — core entities, relationships, and business rules
2. **Map operation tracking** — saga patterns, compensation chains, and audit trails
3. **Define service boundaries** — interfaces, dependencies, and responsibilities

4. **Model state machines** — engine mode, capacity increase requests, and emergency transfers

Important: These diagrams are ~70% complete by design. They intentionally **surface gaps and unresolved design questions** that must be closed before implementation begins. Known gaps are explicitly called out in each diagram.

Diagram Inventory

#	Diagram	Purpose	Status
1	Core Domain Model	Central entities (CRG, Assignment, Policy, QuotaGroup, Snapshot, Increase, Transfer)	70% — entity outlines complete; property types & Cosmos schemas TBD
2	Operation Tracking Model	Saga, OperationRecord, compensation chains, VM_ImpactRecord, audit events	70% — saga pattern defined; compensation lambda structure TBD
3	Service Layer Model	PlacementEngine, Reconciliation, Transfer, Quota, Sharing, Zone services	70% — service boundaries clear; FastAPI contracts & DTOs TBD
4	State Machines	engine_mode, IncreaseRequest lifecycle, Transfer workflow (Tier 1/2/3)	60% — high-level states defined; transition guards & recovery TBD (G-15/B-3 blocker)

Overall Completeness: 68%
Path to 100%: Resolve design gaps (see below), define Cosmos schemas, build FastAPI contracts, close G-14/G-15 blockers

---

Key Design Decisions Captured

From Diagram 1 (Core Domain Model):

✅ **Per-environment CRG structure — regional + per-AZ (v2.4, CAP-023)** — each environment (Prod, NonProd, DR) carries one **regional** CRG (`crg-<env>-<region>-reg`) plus one CRG **per availability zone** (`az1/az2/az3`), extending the earlier three-CRG (Prod/NonProd/DR) model. `CapacityReservationGroup` carries `environment` and `crgScope` (`REGIONAL | ZONAL`). `CustomerRegionAssignment` continues to target the three environments; each environment now resolves to its regional + zonal CRG set. Naming is deterministic per OPS-006/C-12.

✅ **Single governed quota pool per region/quota family (v2.2, QUA-004)** — `QuotaPoolState` covers Prod + NonProd/CVAL + DR together; Prod and DR protected by **logical earmarks** (`prod_reserved_floor` , `dr_earmark_vcpu`), not physical group separation. The earlier `QuotaGroup-Type.PROD` + `QuotaGroupType.NONPROD_DR_SHARED` **two-group** model is retained only as a narrow Azure-limit / Prod-isolation **exception topology** (ADR-002 v2.2).
✅ **v2.2 domain entities added** — `CustomerSeedRecord` (PLC-003, first-placement authority), `Sour-ceDestinationDRIndex` (DR-018, reverse-of-seed driving max-not-sum sizing and standby activation), `CVALEarmarkRecord` (PLC-010, prevents CVAL/DR double-count). See the FDD Section 6 and TDD Section 6/Section 18 (diagram T5, T14) for full schemas.
✅ **v2.4 domain entities added** — `SeedMatrixEntry` (CAP-022, product-team-governed seed matrix initialised at 0, extends CAP-009) and `ReservationEligibility` (CAP-020/CAP-021, Availability-Set-VM ineligibility + deallocate/redeploy-to-AZ onboarding precondition). `CapacityReservation` gains `reser-vationType` (`SEED | ACTIVE` , `isSeed()`) and `placementType` ; reactive SKU/AZ discovery (CAP-024) auto-creates missing reservations against the seed matrix and raises a scope-file governance item. Even ≈ `1/zone_count` zone distribution + rebalancing (PLC-011, tolerance C-13, formula Appendix A. 9). See `uml_01_core_domain_model.md` and TDD Section 6 for full schemas.
✅ **Placement scoring** — `PlacementPolicy` contains weights (alpha, beta, gamma, delta, epsilon) for deterministic placement
✅ **Regional snapshot caching** — `RegionalSnapshot` with 5-min TTL, cached in Redis, persisted to Cosmos
✅ **Sharing relationships** — `SharingRelationship` tracks provider-consumer pairs; 100-consumer hard limit
✅ **Zone mapping** — `ZoneMappingRecord` required for cross-subscription CRG sharing

From Diagram 2 (Operation Tracking Model):

✅ **Saga pattern for all mutations** — `OperationRecord` with compensation chain for rollback
✅ **VM impact audit trail** — `VM_ImpactRecord` created for every Tier 3 VM state change (immutable)
✅ **Incident-driven transfers** — `IncidentRecord` required to enable emergency capacity transfers
✅ **Append-only audit events** — `AuditEvent` for every state change, approval, mutation

From Diagram 3 (Service Layer Model):

✅ **Placement engine** — Applies HC-1..HC-7 hard constraints, then weighted scoring
✅ **Reconciliation loop** — 5-min target interval; debounce guard (30-min cooldown) for auto-increase
✅ **Capacity transfer service** — ONLY callable when `engine_mode == DR_EVENT_ACTIVE`
✅ **Quota validation** — Pre-validates every mutating operation; enforces quota group floors
✅ **VM disassociation service** — Path B (default, VM keeps running) vs. Path A (deallocate, requires restart)
✅ **Zone mapping service** — Translates logical zones for cross-subscription deployments

From Diagram 4 (State Machines):

✅ **Engine mode states** — `STEADY_STATE` , `DR_EVENT_ACTIVE` , `FAILBACK_PENDING` , `MAINTENANCE_MODE`
✅ **Increase request phases** — Phase A (approval-gated), Phase B (auto-approve with headroom check)
✅ **Transfer tiers** — Tier 1 (additive), Tier 2 (quota-neutral), Tier 3 (destructive, dual-approval)
✅ **Compensation on failure** — Every transfer state has defined compensation path

---

Critical Gaps Identified (Must Resolve Before Implementation)

Production Blockers (Cannot Deploy Until Resolved)

Gap ID	Title	Diagram	Impact	Next Step
G-14	Consumer credential model unresolved	2, 3	Tier 3 VM disassociation cannot execute without write access to consumer VMs	Design session: Select UAMI vs. Service Principal; test in consumer subscription; security approval
G-15 / B-3	Engine mode state machine incomplete	4	Cannot implement `engine_mode` transitions without authoritative guards, concurrency controls, recovery rules	Design session: Define transition table, operator API contracts, crash recovery, audit trail

High-Priority Design Gaps (Block Full Implementation)

Gap	Description	Diagram	Resolution Needed
Entity Schemas	Exact Cosmos DB schemas (partition keys, indexes, TTL) TBD	1	For each entity: define PK, partition key, secondary indexes, TTL policy
Compensation Lambdas	Exact structure of compensation functions TBD (Python callable? Azure Function?)	2	Define compensation action signature, retry semantics, idempotency guarantees
FastAPI Contracts	Endpoint signatures, DTOs, auth/RBAC TBD	3	For each service: define request/response models, validation rules, RBAC roles
Placement Hard Constraints	HC-1..HC-7 exact predicate logic TBD	3	Define each constraint as testable predicate; validate against POC workbook
Quota Formula Validation	Risk of double-counting (R-39); exact Azure Quota API semantics TBD	3	Test against live Azure quota API; document exact formula per SKU family
Dual Approval Workflow	Tier 2/3 require dual approval; exact mechanism TBD	4	ITSM integration? Inline API? Approval timeout policy?
Tier Escalation Logic	Does Tier 1 failure auto-escalate to Tier 2?	4	Define decision tree; operator-gated vs. auto-escalate
VMSS Phase 1 Limitation	How to detect and reject VMSS for Tier 3?	3, 4	Resource type check; error message; Phase 2 roadmap

Medium-Priority Design Questions (Refinement Needed)

- **Concurrent placement race (B-7):** How to prevent two placements from selecting same region before either commits?
- **Snapshot staleness:** ARG can be minutes behind ARM; acceptable staleness threshold TBD

- **Reconciliation drift policy:** When drift detected, auto-revert? Alert-only? Maintenance mode?
- **Retry strategy:** Max retries, backoff algorithm, dead-letter handling for failed operations
- **Compensation rollback:** If compensation itself fails, manual SOP TBD
- **Auto-increase Phase A → B:** Exact criteria for auto-approve vs. approval-gated

---

How to Use These Diagrams

1. As a Design Review Tool

- **For stakeholders:** Visual overview of system architecture and data flows
- **For architects:** Identifies relationships, dependencies, and design patterns
- **For engineers:** Surfaces exact questions that must be answered before coding

2. As a Gap-Driven Design Session Roadmap

Each diagram explicitly calls out **“Known Gaps”** sections. Use these to drive focused design sessions:

Session 1: G-15/B-3 Engine Mode State Machine
- Input: Diagram 4, Section 4.1
- Output: Authoritative transition table, operator API contracts, concurrency controls
- Acceptance: POC-46 unblocked, B-3 resolved

Session 2: G-14 Consumer Credential Model
- Input: Diagram 2 (VM_ImpactRecord), Diagram 3 (VMDisassociationService)
- Output: Selected credential model (UAMI vs. SP), custom RBAC role, customer consent workflow
- Acceptance: POC-50 Tier 3 steps unblocked, security approval granted

Session 3: Entity Schemas & Cosmos DB Design
- Input: Diagram 1 (all entities)
- Output: Complete Cosmos schemas with partition keys, indexes, TTL
- Acceptance: Can provision Cosmos containers; schema versioning strategy defined

Session 4: Service Contracts & FastAPI Routes
- Input: Diagram 3 (all services)
- Output: Request/response DTOs, endpoint signatures, RBAC roles
- Acceptance: OpenAPI spec generated; can build API stubs

3. As a Living Design Document

- **Update as decisions are made:** When G-14 resolved, update Diagram 2/3 with selected credential model
- **Refine property types:** As Cosmos schemas finalized, update Diagram 1 with exact types
- **Version control:** Commit diagrams to Git; review in PRs before implementation

---

Relationships to Other Artifacts

Artifact	Relationship
Production Readiness Review	Identifies gaps G-14, G-15, B-3, B-7; UML diagrams visualize these blockers
POC Test Workbook	UML entities map to POC test scenarios (e.g., `CustomerRegionAssignment` → POC-34..41)
Architecture Diagrams Deck	High-level visual flows; UML provides detailed class/service structure
Test Automation Suite	UML services map to test groups (e.g., `SharingManagementService` → G2 tests)

Diagram Source Files

All diagrams are stored in `/Design/` as Markdown + Mermaid:

- `uml_01_core_domain_model.md` — Entities, relationships, enumerations
- `uml_02_operation_tracking_model.md` — Saga, compensation, audit
- `uml_03_service_layer_model.md` — Services, interfaces, dependencies
- `uml_04_state_machines.md` — Engine mode, IncreaseRequest, Transfer workflows

Rendering: GitHub renders Mermaid natively in Markdown preview. For presentation, convert to PNG/SVG via `mmdc` CLI or Mermaid Live Editor.

Next Steps

Immediate (Week 1):

1. **G-15/B-3 Design Session** — Resolve engine mode state machine (unblocks POC-46, resolves B-3)
2. **G-14 Design Session** — Select credential model for Tier 3 VM disassociation (unblocks POC-50)

Short-Term (Week 2-3):

1. **Entity Schema Definition** — Complete Cosmos DB schemas for all Diagram 1 entities
2. **Service Contract Definition** — FastAPI DTOs, endpoint signatures, RBAC roles for all Diagram 3 services

Medium-Term (Week 4-6):

1. **Compensation Lambda Implementation** — Build saga framework with retry, idempotency, dead-letter
2. **State Machine Implementation** — Code engine mode, IncreaseRequest, Transfer workflows with tests

Long-Term (Post-MVP):

1. **VMSS Phase 2 Support** — Extend VM disassociation to handle VMSS
2. **Auto-Increase Phase B** — Implement self-managed auto-approve logic
3. **Multi-SKU Placement** — Extend placement formulas for SKU-dimensional demand

---

Validation Checklist

Before marking diagrams as “implementation-ready” (100% complete):

- [ ] All entity properties have exact types (String, Int, DateTime, etc.)
- [ ] All Cosmos DB partition keys, indexes, TTL policies defined
- [ ] All service methods have FastAPI DTOs (request/response models)
- [ ] All state machine transitions have guards, events, approver requirements
- [ ] All compensation actions have idempotent lambda signatures
- [ ] G-14 and G-15/B-3 production blockers resolved
- [ ] All “TBD” markers in diagrams replaced with concrete decisions
- [ ] Diagrams reviewed and approved by architecture council

Conclusion

These UML class diagrams provide a **solid 70% foundation** for ACRME implementation. They capture known design decisions, visualize relationships, and — most importantly — **explicitly surface the 30% of design work still required**.

The path to 100% is clear:
1. Resolve production blockers (G-14, G-15/B-3)
2. Define complete schemas and contracts
3. Build saga framework and state machines
4. Validate against POC workbook

Use these diagrams as living design tools, not static documentation. Update them as decisions are made, and they will remain the authoritative source of truth for ACRME’s domain model and architecture.